

calculate_preTARE_am_kpis_sparkGap_COP_20April2026

April 21, 2026

Pre-TARE Adoption KPIs: Spark Gap, Thermal COP, Break-Even COP

Author: Jordan M. Joseph, PhD — Carnegie Mellon University

Computes adoption economics metrics that can be calculated directly from EUSS data and EIA fuel prices, without requiring a TARE model run.

Workflow: Load EUSS data → spark gap → thermal COP & AFUE → break-even COP → maps

Batch Mode: Set `input_measure_package` before calling via `%run -i` to skip prompts.

See `README_adoption_kpis.md` for methodology notes and design decisions.

Step 0: Imports and Configuration

```
[1]: import os
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

from config import PROJECT_ROOT
from cmu_tare_model.constants import ALLOWED_HOUSING_TYPES, VALID_MENU_MPS, _
    ↪ VERBOSE

from cmu_tare_model.adoption_kpis.kpi_functions import (
    mp_to_upgrade,
    load_euss_baseline,
    load_euss_upgrade,
    calculate_price_ratios,
    compute_thermal_cop,           # renamed from compute_thermal_cop_by_state
    compute_breakeven_cop,
    compute_spark_gap_metrics,
    iecc_to_cz_group,             # new - climate zone mapping
    COP_BENCHMARK_RANGES,        # new - validation benchmark ranges
    FUEL_PRICES_PATH,
    SHAPEFILE_PATH,
```

```

    HEATING_LOAD_COL,
    CLIMATE_ZONE_COL,          # new - column name constant
)
from cmu_tare_model.adoption_kpis.visualize_geospatial_data import (
    prepare_state_geodataframe,
    create_choropleth_map,
)

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 60)

print(" Imports loaded")
print(f"Project root: {PROJECT_ROOT}")

```

Project root directory: c:\users\jorda\desktop\projects\cmu-tare-model

Imports loaded

Project root: c:\users\jorda\desktop\projects\cmu-tare-model

Step 0b: Measure Package Selection

```

[2]: SELECTABLE_MPS = [mp for mp in VALID_MENU_MPS if mp != 0]

try:
    _ = input_measure_package
    batch_mode = True
    selected_mps = [int(input_measure_package)]
    print(f"BATCH MODE: Running for MP{selected_mps[0]}")
except NameError:
    batch_mode = False
    print(f"Available measure packages: {SELECTABLE_MPS}")
    mp_input = input("Enter MP numbers (comma-separated, or 'all'): ").strip()
    if mp_input.lower() == 'all':
        selected_mps = SELECTABLE_MPS
    else:
        selected_mps = [int(x.strip()) for x in mp_input.split(',') if x.
↪strip().isdigit()]
        selected_mps = [mp for mp in selected_mps if mp in SELECTABLE_MPS]
    if not selected_mps:
        selected_mps = [4]
        print("No valid MPs selected. Defaulting to MP4.")

print(f"\nSelected measure packages: {selected_mps}")

```

Available measure packages: [3, 4, 8]

Selected measure packages: [3, 4]

Step 1: Load EUSS Data

```
[3]: print("=" * 80)
      print("STEP 1: LOAD EUSS DATA")
      print("=" * 80)

      df_baseline = load_euss_baseline()
      print(f"  Baseline: {len(df_baseline):,} occupied SF homes")

      upgrade_data = {}
      for mp in selected_mps:
          upgrade_name = mp_to_upgrade(mp)
          print(f"\nLoading MP{mp} ({upgrade_name})...")
          upgrade_data[mp] = load_euss_upgrade(upgrade_name)
          print(f"  MP{mp}: {len(upgrade_data[mp]):,} applicable homes")

      print(f"\n STEP 1 COMPLETE")
```

=====

STEP 1: LOAD EUSS DATA

=====

Loading baseline from: c:\users\jorda\desktop\projects\cmu-tare-model\cmu_tare_model\data\euss_data\resstock_amy2018_release_1.1\national\csv\baseline_metadata_and_annual_results.csv

After occupancy filter: 482,597 / 548,916

After housing type filter (['Single-Family Attached', 'Single-Family Detached']): 331,531

Baseline: 331,531 occupied SF homes

Loading MP3 (upgrade03)...

Loading upgrade03 from: c:\users\jorda\desktop\projects\cmu-tare-model\cmu_tare_model\data\euss_data\resstock_amy2018_release_1.1\national\csv\upgrade03_metadata_and_annual_results.csv

After occupancy filter: 482,597 / 548,916

After housing type filter: 331,531

After applicability filter: 331,526

MP3: 331,526 applicable homes

Loading MP4 (upgrade04)...

Loading upgrade04 from: c:\users\jorda\desktop\projects\cmu-tare-model\cmu_tare_model\data\euss_data\resstock_amy2018_release_1.1\national\csv\upgrade04_metadata_and_annual_results.csv

After occupancy filter: 482,597 / 548,916

After housing type filter: 331,531

After applicability filter: 331,526

MP4: 331,526 applicable homes

STEP 1 COMPLETE

Step 2: Fuel Price Ratios (Spark Gap)

```
[4]: print("==== STEP 2: FUEL PRICE RATIOS (CSV, nominal) =====")

# --- 2022 single-year ---
df_prices_2022 = calculate_price_ratios(FUEL_PRICES_PATH, year=2022)
print(f" 2022 prices loaded for {len(df_prices_2022)} states")
print(f" Spark Gap - Mean: {df_prices_2022['spark_gap'].mean():.2f}, "
      f"Range: {df_prices_2022['spark_gap'].min():.2f}-"
      f"{df_prices_2022['spark_gap'].max():.2f}")

# --- 2024 single-year ---
df_prices_csv = calculate_price_ratios(FUEL_PRICES_PATH, year=2024)
print(f"\n 2024 prices loaded for {len(df_prices_csv)} states")
print(f" Spark Gap - Mean: {df_prices_csv['spark_gap'].mean():.2f}, "
      f"Range: {df_prices_csv['spark_gap'].min():.2f}-"
      f"{df_prices_csv['spark_gap'].max():.2f}")

# --- 5-year average (2020-2024) ---
df_prices_5yr = calculate_price_ratios(FUEL_PRICES_PATH, year=list(range(2020, 2025)))
print(f"\n 5-year avg (2020-2024) loaded for {len(df_prices_5yr)} states")
print(f" Spark Gap - Mean: {df_prices_5yr['spark_gap'].mean():.2f}, "
      f"Range: {df_prices_5yr['spark_gap'].min():.2f}-"
      f"{df_prices_5yr['spark_gap'].max():.2f}")

# --- 10-year average (2015-2024) ---
df_prices_10yr = calculate_price_ratios(FUEL_PRICES_PATH, year=list(range(2015, 2025)))
print(f"\n 10-year avg (2015-2024) loaded for {len(df_prices_10yr)} states")
print(f" Spark Gap - Mean: {df_prices_10yr['spark_gap'].mean():.2f}, "
      f"Range: {df_prices_10yr['spark_gap'].min():.2f}-"
      f"{df_prices_10yr['spark_gap'].max():.2f}")

print(f"\n STEP 2 COMPLETE")
```

==== STEP 2: FUEL PRICE RATIOS (CSV, nominal) =====

2022 prices loaded for 51 states

Spark Gap - Mean: 3.15, Range: 1.61-6.34

2024 prices loaded for 51 states

Spark Gap - Mean: 3.55, Range: 1.70-6.44

5-year avg (2020-2024) loaded for 51 states

Spark Gap - Mean: 3.41, Range: 1.67-6.32

10-year avg (2015-2024) loaded for 51 states
Spark Gap - Mean: 3.55, Range: 1.67-6.27

STEP 2 COMPLETE

```
[5]: # Build summary table: 2024 prices + spark gaps for 2022, 2024, 5yr, 10yr
df_summary = df_prices_csv[['state', 'state_name', 'elec_price_mmbtu',
    ↪ 'gas_price_mmbtu']].copy()
df_summary = df_summary.rename(columns={
    'elec_price_mmbtu': 'elec$/mmbtu_2024',
    'gas_price_mmbtu': 'gas$/mmbtu_2024',
})

df_summary = df_summary.merge(
    df_prices_2022[['state', 'spark_gap']].rename(columns={'spark_gap':
    ↪ 'spark_gap_2022'}),
    on='state', how='inner',
)
df_summary = df_summary.merge(
    df_prices_csv[['state', 'spark_gap']].rename(columns={'spark_gap':
    ↪ 'spark_gap_2024'}),
    on='state', how='inner',
)
df_summary = df_summary.merge(
    df_prices_5yr[['state', 'spark_gap']].rename(columns={'spark_gap':
    ↪ 'spark_gap_5yr_avg'}),
    on='state', how='inner',
)
df_summary = df_summary.merge(
    df_prices_10yr[['state', 'spark_gap']].rename(columns={'spark_gap':
    ↪ 'spark_gap_10yr_avg'}),
    on='state', how='inner',
)

# Sort by 2024 spark gap for display
df_summary = df_summary.sort_values('spark_gap_2024', ascending=False).
    ↪ reset_index(drop=True)

display_cols = [
    'state', 'state_name',
    'elec$/mmbtu_2024', 'gas$/mmbtu_2024',
    'spark_gap_2022', 'spark_gap_2024', 'spark_gap_5yr_avg',
    ↪ 'spark_gap_10yr_avg',
]

print("=" * 100)
```

```

print("SPARK GAP SUMMARY: TOP 5 & BOTTOM 5 STATES")
print("=" * 100)
print(f"\n{'Elec & Gas prices shown as 2024 nominal $/MMBTU'}")
print(f"{'Spark Gap = Electricity Price / Natural Gas Price'}\n")

print("--- Top 5 (highest spark gap, most favorable for gas heating) ---")
print(df_summary[display_cols].head(5).to_string(index=False))

print("\n--- Bottom 5 (lowest spark gap, most favorable for heat pumps) ---")
print(df_summary[display_cols].tail(5).to_string(index=False))

print(f"\n--- National Summary ---")
for label, col in [('2022', 'spark_gap_2022'), ('2024', 'spark_gap_2024'),
                  ('5yr avg', 'spark_gap_5yr_avg'), ('10yr avg', 'spark_gap_10yr_avg')]:
    vals = df_summary[col]
    print(f" {label:>8}: Mean={vals.mean():.2f}, Median={vals.median():.2f}, "
          f"Range=[{vals.min():.2f}, {vals.max():.2f}]")

```

```

=====
=====
SPARK GAP SUMMARY: TOP 5 & BOTTOM 5 STATES
=====
=====

```

Elec & Gas prices shown as 2024 nominal \$/MMBTU
 Spark Gap = Electricity Price / Natural Gas Price

```

--- Top 5 (highest spark gap, most favorable for gas heating) ---
state state_name elec_$/mmbtu_2024 gas_$/mmbtu_2024 spark_gap_2022
spark_gap_2024 spark_gap_5yr_avg spark_gap_10yr_avg
  AK      Alaska      72.74      11.30      6.34
6.44      6.32      6.27
  MI      Michigan      56.56      10.37      4.80
5.46      5.36      5.45
  CT Connecticut      84.26      16.21      4.08
5.20      4.59      4.63
  WI      Wisconsin      50.35      9.77      4.19
5.15      4.81      5.02
  CA      California      93.69      18.44      3.90
5.08      4.34      4.41

```

```

--- Bottom 5 (lowest spark gap, most favorable for heat pumps) ---
state state_name elec_$/mmbtu_2024 gas_$/mmbtu_2024 spark_gap_2022
spark_gap_2024 spark_gap_5yr_avg spark_gap_10yr_avg
  MO      Missouri      37.84      16.24      2.50
2.33      2.62      2.84
  AR      Arkansas      36.11      15.50      2.16

```

2.33		2.34		2.45		
AZ	Arizona		43.70		19.72	2.22
2.22		2.35		2.40		
LA	Louisiana		34.38		15.78	2.48
2.18		2.35		2.42		
FL	Florida		41.44		24.44	1.61
1.70		1.67		1.67		

--- National Summary ---

2022: Mean=3.15, Median=3.07, Range=[1.61, 6.34]

2024: Mean=3.55, Median=3.41, Range=[1.70, 6.44]

5yr avg: Mean=3.41, Median=3.28, Range=[1.67, 6.32]

10yr avg: Mean=3.55, Median=3.43, Range=[1.67, 6.27]

Step 3: Thermal COP and Baseline AFUE

```
[6]: cop_results = {}

for mp in selected_mps:
    print(f"\n{'=' * 80}")
    print(f"STEP 3: THERMAL COP & BASELINE AFUE (MP{mp}, Natural Gas homes)")
    print(f"\n{'=' * 80}")

    cop_results[mp] = compute_thermal_cop(
        df_baseline, upgrade_data[mp],
        group_cols=['state'],
        fuel_filter='Natural Gas',
        verbose=True,
    )

    df_cop_mp = cop_results[mp]
    print(f"\n--- Top 5 by COP (MP{mp}) ---")
    print(df_cop_mp.sort_values('thermal_cop', ascending=False).head(5)[
        ['state', 'thermal_cop', 'baseline_afue', 'fans_pumps_pct',
↪ 'home_count']
    ].to_string(index=False))

    print(f"\n--- Bottom 5 by COP (MP{mp}) ---")
    print(df_cop_mp.sort_values('thermal_cop', ascending=True).head(5)[
        ['state', 'thermal_cop', 'baseline_afue', 'fans_pumps_pct',
↪ 'home_count']
    ].to_string(index=False))

    # Validation: heating load consistency
    common_ids = df_baseline.index.intersection(upgrade_data[mp].index)
    Q_baseline = df_baseline.loc[common_ids, HEATING_LOAD_COL].fillna(0)
    Q_upgrade = upgrade_data[mp].loc[common_ids, HEATING_LOAD_COL].fillna(0)
```

```

mask = Q_baseline > 0
pct_diff = ((Q_upgrade[mask] - Q_baseline[mask]) / Q_baseline[mask]).
↳median()
print(f"\nHeating load consistency: {pct_diff:.1%} median difference_
↳(expect ±5-10%)")

primary_mp = selected_mps[0]
df_cop = cop_results[primary_mp]
df_upgrade_primary = upgrade_data[primary_mp]
print(f"\n STEP 3 COMPLETE - Primary MP: {primary_mp}")

```

=====

STEP 3: THERMAL COP & BASELINE AFUE (MP3, Natural Gas homes)

=====

Matched homes (baseline upgrade): 331,526
 Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)
 Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by state) ---

Groups: 49
 Mean: 2.21
 Median: 2.20
 Range: 1.53 - 3.05
 All groups within expected COP range (1.5-5.0)

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.80
 Median: 0.80
 Range: 0.75 - 0.82

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 3.8%
 Range: 1.7% - 5.8%

--- Top 5 by COP (MP3) ---

state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
CA	3.054755	0.801801	4.001501	24571
FL	2.893052	0.794453	5.682569	1084
LA	2.758595	0.799742	5.761571	2139
TX	2.748806	0.804379	5.035227	12362
DC	2.715392	0.752035	4.055966	338

--- Bottom 5 by COP (MP3) ---

state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
ND	1.527465	0.803502	1.730071	402
MN	1.589963	0.801746	1.937747	4687

ME	1.647215	0.819947	1.661832	61
SD	1.673837	0.797856	2.435159	520
WI	1.699153	0.804310	2.207530	4759

Heating load consistency: 0.1% median difference (expect ± 5 -10%)

=====

STEP 3: THERMAL COP & BASELINE AFUE (MP4, Natural Gas homes)

=====

Matched homes (baseline upgrade): 331,526
 Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)
 Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by state) ---

Groups: 49

Mean: 3.89

Median: 3.98

Range: 2.15 - 5.72

8 groups with suspicious COP (<1.5 or >5.0):

AL: 5.06

CA: 5.72

FL: 5.70

GA: 5.06

LA: 5.35

MS: 5.02

SC: 5.06

TX: 5.30

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.80

Median: 0.80

Range: 0.75 - 0.82

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 4.5%

Range: 2.4% - 5.7%

--- Top 5 by COP (MP4) ---

state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
CA	5.719542	0.801801	5.075078	24571
FL	5.697018	0.794453	5.459375	1084
LA	5.345519	0.799742	5.496742	2139
TX	5.303021	0.804379	5.558138	12362
AL	5.058210	0.805110	5.564781	1853

--- Bottom 5 by COP (MP4) ---

state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
ND	2.149135	0.803502	2.371268	402

MN	2.359931	0.801746	2.772684	4687
ME	2.444452	0.819947	2.846700	61
SD	2.502694	0.797856	3.040386	520
WI	2.623592	0.804310	3.174381	4759

Heating load consistency: 5.2% median difference (expect $\pm 5\text{-}10\%$)

STEP 3 COMPLETE - Primary MP: 3

Task B: COP Direction Check (Zero-Baseline Filter Validation)

Validates that the zero-baseline filter (excluding homes with no prior heating system) lowers COP as expected. Compares fixed (default) vs unfixed (`require_baseline_heating=False`) results per state.

```
[7]: print("=" * 80)
print("TASK B: COP DIRECTION CHECK (zero-baseline filter validation)")
print("=" * 80)

for mp in selected_mps:
    print(f"\n--- MP{mp} ---")

    # Unfixed: include homes with zero baseline heating (old behavior)
    cop_unfixed = compute_thermal_cop(
        df_baseline, upgrade_data[mp],
        group_cols=['state'],
        fuel_filter='Natural Gas',
        require_baseline_heating=False,
        verbose=False,
    )

    # Fixed: cop_results[mp] already computed with
    ↪ require_baseline_heating=True (default)
    cop_fixed = cop_results[mp]

    # Merge for comparison
    df_compare = cop_unfixed[['state', 'thermal_cop']].rename(
        columns={'thermal_cop': 'cop_unfixed'})
    ).merge(
        cop_fixed[['state', 'thermal_cop']].rename(
            columns={'thermal_cop': 'cop_fixed'})
        ),
        on='state', how='outer',
    )
    df_compare['delta'] = df_compare['cop_fixed'] - df_compare['cop_unfixed']
    df_compare['direction_ok'] = df_compare['cop_fixed'] <= ↪
    ↪ df_compare['cop_unfixed']
```

```

    print(f"\n{'State':<6} {'COP_unfixed':>12} {'COP_fixed':>10} {'Delta':>8}␣
↪{'Dir OK?':>8}")
    print("-" * 48)
    for _, r in df_compare.sort_values('state').iterrows():
        print(f"{r['state']:<6} {r['cop_unfixed']:>12.3f} {r['cop_fixed']:>10.
↪3f} "
              f"{r['delta']:>+8.3f} {' ' if r['direction_ok'] else ' ':>8}")

    n_pass = df_compare['direction_ok'].sum()
    n_total = len(df_compare)
    n_fail = n_total - n_pass
    print(f"\nSummary: {n_pass}/{n_total} states pass direction check")
    if n_fail > 0:
        fails = df_compare[~df_compare['direction_ok']]
        print(f" {n_fail} state(s) where fixed COP > unfixed (investigate):")
        for _, r in fails.iterrows():
            print(f"      {r['state']}: unfixed={r['cop_unfixed']:.3f}, "
                  f"fixed={r['cop_fixed']:.3f}, delta={r['delta']:+.3f}")
    else:
        print(" All states: fixed COP   unfixed COP (filter lowered COP as␣
↪expected)")

print(f"\n TASK B COMPLETE")

```

```

=====
TASK B: COP DIRECTION CHECK (zero-baseline filter validation)
=====

```

--- MP3 ---

State	COP_unfixed	COP_fixed	Delta	Dir OK?
AL	2.608	2.608	+0.000	
AR	2.499	2.499	+0.000	
AZ	2.587	2.588	+0.001	
CA	3.054	3.055	+0.001	
CO	1.952	1.952	+0.000	
CT	2.086	2.086	+0.000	
DC	2.715	2.715	+0.000	
DE	2.315	2.315	+0.000	
FL	2.893	2.893	+0.000	
GA	2.599	2.599	+0.000	
IA	1.763	1.763	+0.000	
ID	2.155	2.155	+0.000	
IL	1.946	1.946	+0.000	
IN	1.972	1.972	+0.000	

KS	2.201	2.201	+0.000
KY	2.290	2.290	+0.000
LA	2.759	2.759	+0.000
MA	2.012	2.012	+0.000
MD	2.399	2.399	+0.000
ME	1.647	1.647	+0.000
MI	1.786	1.786	+0.000
MN	1.590	1.590	+0.000
MO	2.228	2.228	+0.000
MS	2.584	2.584	+0.000
MT	1.892	1.892	+0.000
NC	2.471	2.471	+0.000
ND	1.527	1.527	+0.000
NE	1.898	1.898	+0.000
NH	1.854	1.854	+0.000
NJ	2.294	2.294	+0.000
NM	2.384	2.384	+0.000
NV	2.491	2.491	+0.001
NY	1.974	1.974	+0.000
OH	1.990	1.990	+0.000
OK	2.493	2.493	+0.000
OR	2.619	2.619	+0.000
PA	2.020	2.020	+0.000
RI	1.937	1.937	+0.000
SC	2.603	2.603	+0.000
SD	1.674	1.674	+0.000
TN	2.408	2.408	+0.000
TX	2.749	2.749	+0.000
UT	2.163	2.163	+0.000
VA	2.380	2.380	+0.000
VT	1.731	1.731	+0.000
WA	2.510	2.510	+0.000
WI	1.699	1.699	+0.000
WV	2.051	2.051	+0.000
WY	1.977	1.977	+0.000

Summary: 44/49 states pass direction check

5 state(s) where fixed COP > unfixed (investigate):

AZ: unfixed=2.587, fixed=2.588, delta=+0.001

CA: unfixed=3.054, fixed=3.055, delta=+0.001

FL: unfixed=2.893, fixed=2.893, delta=+0.000

NV: unfixed=2.491, fixed=2.491, delta=+0.001

TX: unfixed=2.749, fixed=2.749, delta=+0.000

--- MP4 ---

State	COP_unfixed	COP_fixed	Delta	Dir OK?

AL	5.058	5.058	+0.000
AR	4.830	4.830	+0.000
AZ	4.685	4.685	-0.000
CA	5.720	5.720	-0.000
CO	3.075	3.075	+0.000
CT	3.595	3.595	+0.000
DC	4.981	4.981	+0.000
DE	4.261	4.261	+0.000
FL	5.697	5.697	+0.000
GA	5.057	5.057	+0.000
IA	2.850	2.850	+0.000
ID	3.594	3.594	+0.000
IL	3.302	3.302	+0.000
IN	3.337	3.337	+0.000
KS	3.980	3.980	+0.000
KY	4.146	4.146	+0.000
LA	5.346	5.346	+0.000
MA	3.375	3.375	+0.000
MD	4.428	4.428	+0.000
ME	2.444	2.444	+0.000
MI	2.829	2.829	+0.000
MN	2.360	2.360	+0.000
MO	4.083	4.083	+0.000
MS	5.024	5.024	+0.000
MT	2.846	2.846	+0.000
NC	4.724	4.724	+0.000
ND	2.149	2.149	+0.000
NE	3.180	3.180	+0.000
NH	3.056	3.056	+0.000
NJ	4.131	4.131	+0.000
NM	4.107	4.107	+0.000
NV	4.481	4.481	-0.000
NY	3.202	3.202	+0.000
OH	3.343	3.343	+0.000
OK	4.800	4.800	+0.000
OR	4.661	4.661	+0.000
PA	3.342	3.342	+0.000
RI	3.167	3.167	+0.000
SC	5.057	5.057	+0.000
SD	2.503	2.503	+0.000
TN	4.567	4.567	+0.000
TX	5.303	5.303	+0.000
UT	3.698	3.698	+0.000
VA	4.460	4.460	+0.000
VT	2.676	2.676	+0.000
WA	4.220	4.220	+0.000
WI	2.624	2.624	+0.000
WV	3.436	3.436	+0.000

WY 2.960 2.960 +0.000

Summary: 49/49 states pass direction check

All states: fixed COP unfixed COP (filter lowered COP as expected)

TASK B COMPLETE

Task C: Climate Zone Benchmark Validation

Aggregates thermal COP by IECC climate zone group and validates against literature-derived benchmark ranges (COP_BENCHMARK_RANGES). Also produces a state $\times$ climate zone cross-tab with a PA spot check.

```
[8]: print("=" * 80)
print("TASK C: CLIMATE ZONE BENCHMARK VALIDATION")
print("=" * 80)

for mp in selected_mps:
    mp_key = f'mp{mp}'
    print(f"\n{' ' * 60}")
    print(f"MP{mp} - Climate Zone Group Aggregation")
    print(f"{' ' * 60}")

    # --- C1: COP by climate zone group ---
    df_cop_cz = compute_thermal_cop(
        df_baseline, upgrade_data[mp],
        group_cols=['cz_group'],
        fuel_filter='Natural Gas',
        verbose=True,
    )

    # --- C2: COP by state  $\times$  climate zone group ---
    df_cop_state_cz = compute_thermal_cop(
        df_baseline, upgrade_data[mp],
        group_cols=['state', 'cz_group'],
        fuel_filter='Natural Gas',
        verbose=True,
    )

    # --- C3: Validate against COP_BENCHMARK_RANGES ---
    print(f"\n--- Benchmark Validation (MP{mp}) ---")
    print(f"{'CZ Group':<10} {'Label':<18} {'COP':>6} {'Expected':>14} {'Pass':>6}")
    print("-" * 58)
    n_checked = 0
    n_passed = 0
    for cz_grp, bench in COP_BENCHMARK_RANGES.items():
```

```

row = df_cop_cz[df_cop_cz['cz_group'] == cz_grp]
if len(row) == 0:
    print(f"{cz_grp:<10} {bench['label']:<18} {'N/A':>6} {'':<14}␣
↳{'SKIP':>6}")
    continue
cop_val = row['thermal_cop'].values[0]
if mp_key in bench:
    lo, hi = bench[mp_key]
    in_range = lo <= cop_val <= hi
    n_checked += 1
    n_passed += int(in_range)
    range_str = f"[{lo:.1f}, {hi:.1f}]"
    print(f"{cz_grp:<10} {bench['label']:<18} {cop_val:>6.2f}␣
↳{range_str:>14} "
        f"{' ' if in_range else '':>6}")
else:
    print(f"{cz_grp:<10} {bench['label']:<18} {cop_val:>6.2f} {'no␣
↳bench':>14} {'-':>6}")

print(f"\nOverall: {n_passed}/{n_checked} climate zone groups within␣
↳expected range")

# --- C4: PA spot check ---
pa_rows = df_cop_state_cz[df_cop_state_cz['state'] == 'PA']
if len(pa_rows) > 0:
    print(f"\n--- PA Spot Check (MP{mp}) ---")
    # ASSUMPTION: PA is primarily CZ 4-5
    pa_ranges = {'mp3': (1.8, 2.4), 'mp4': (2.5, 3.4)}
    for _, pa_row in pa_rows.iterrows():
        cop_val = pa_row['thermal_cop']
        cz = pa_row['cz_group']
        if mp_key in pa_ranges:
            lo, hi = pa_ranges[mp_key]
            ok = lo <= cop_val <= hi
            print(f"  PA (CZ {cz}): COP = {cop_val:.2f}, "
                  f"expected [{lo:.1f}, {hi:.1f}] → {' PASS' if ok else '␣
↳FAIL'}")
        else:
            print(f"  PA (CZ {cz}): COP = {cop_val:.2f} (no PA benchmark␣
↳for {mp_key})")
    else:
        print("\n PA not found in state × CZ cross-tab")

print(f"\n TASK C COMPLETE")

```

=====

TASK C: CLIMATE ZONE BENCHMARK VALIDATION

=====

MP3 - Climate Zone Group Aggregation

Matched homes (baseline upgrade): 331,526
Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)
Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by cz_group) ---

Groups: 3
Mean: 2.15
Median: 2.05
Range: 1.67 - 2.74
All groups within expected COP range (1.5-5.0)

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.80
Median: 0.80
Range: 0.80 - 0.81

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 3.4%
Range: 2.2% - 4.8%
Matched homes (baseline upgrade): 331,526
Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)
Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by state × cz_group) ---

Groups: 72
Mean: 2.22
Median: 2.20
Range: 1.53 - 3.15
All groups within expected COP range (1.5-5.0)

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.81
Median: 0.81
Range: 0.75 - 0.85

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 3.9%
Range: 1.6% - 6.2%

--- Benchmark Validation (MP3) ---

CZ Group	Label	COP	Expected	Pass
1-3	Warm (CZ 1-3)	2.74	[2.4, 3.2]	

4-5	Mixed (CZ 4-5)	2.05	[2.0, 2.8]
6-7	Cold (CZ 6-7)	1.67	[1.6, 2.4]

Overall: 3/3 climate zone groups within expected range

--- PA Spot Check (MP3) ---

PA (CZ 4-5): COP = 2.03, expected [1.8, 2.4] → PASS

PA (CZ 6-7): COP = 1.63, expected [1.8, 2.4] → FAIL

MP4 - Climate Zone Group Aggregation

Matched homes (baseline upgrade): 331,526

Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)

Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by cz_group) ---

Groups: 3

Mean: 3.76

Median: 3.47

Range: 2.51 - 5.30

1 groups with suspicious COP (<1.5 or >5.0):

1-3: 5.30

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.80

Median: 0.80

Range: 0.80 - 0.81

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 4.2%

Range: 3.0% - 5.5%

Matched homes (baseline upgrade): 331,526

Filtered to 'Natural Gas': 180,939 / 331,526 homes (54.6%)

Excluded 683 homes with zero baseline heating (180,256 remaining)

--- Thermal COP Summary (Natural Gas, by state × cz_group) ---

Groups: 72

Mean: 3.89

Median: 3.74

Range: 2.15 - 6.31

13 groups with suspicious COP (<1.5 or >5.0):

AL, 1-3: 5.06

AR, 1-3: 5.02

AZ, 1-3: 5.82

CA, 1-3: 5.96

FL, 1-3: 5.70

GA, 1-3: 5.06

GA, 4-5: 5.03
 LA, 1-3: 5.35
 MS, 1-3: 5.02
 NV, 1-3: 6.31
 SC, 1-3: 5.06
 TX, 1-3: 5.35
 UT, 1-3: 5.35

--- Baseline AFUE Summary (Natural Gas) ---

Mean: 0.81
 Median: 0.81
 Range: 0.75 - 0.85

--- Fan/Pump Energy as % of Total HP Electricity ---

Mean: 4.5%
 Range: 2.4% - 5.8%

--- Benchmark Validation (MP4) ---

CZ Group	Label	COP	Expected	Pass
1-3	Warm (CZ 1-3)	5.30	[3.0, 4.2]	
4-5	Mixed (CZ 4-5)	3.47	[2.5, 3.5]	
6-7	Cold (CZ 6-7)	2.51	[2.0, 3.0]	

Overall: 2/3 climate zone groups within expected range

--- PA Spot Check (MP4) ---

PA (CZ 4-5): COP = 3.39, expected [2.5, 3.4] → PASS
 PA (CZ 6-7): COP = 2.28, expected [2.5, 3.4] → FAIL

TASK C COMPLETE

Task D: Break-Even COP Cross-Validation (Jenkins et al.)

Rich cross-validation against Jenkins Heat Pump Map reference values. For each reference state: compares thermal COP, spark gap, computed break-even COP at 90% AFUE, and Jenkins reference. Determines whether the heat pump beats break-even and whether the computed value matches Jenkins within strict (± 0.05) and relaxed (± 0.50) tolerances.

Known discrepancy sources: 1. Gas heat content — Jenkins: 1020 BTU/cf; ours: 1038 BTU/cf (EIA average) 2. Price year — Jenkins: earlier year; ours: 2024 EIA nominal

```
[9]: print("=" * 80)
      print(f"TASK D: BREAK-EVEN COP CROSS-VALIDATION (Jenkins et al.,
            ↪MP{primary_mp})")
      print("=" * 80)
```

```

# Jenkins Heat Pump Map break-even COP at 90% AFUE reference values
# Source: Jenkins et al.
# ASSUMPTION: Jenkins assumes 1020 BTU/cf gas heat content; we use 1038 BTU/cf
# (current EIA average). This ~1.8% difference propagates into spark gap and
# break-even COP. Our prices are now 2024 EIA nominal (same as Jenkins).
jenkins_ref = {
    'FL': 1.50, 'PA': 3.51, 'MN': 3.90,
    'AK': 5.69, 'CA': 4.49, 'MA': 3.60,
}

# Use state-level COP from cop_results and spark gap from current prices
df_prices_for_d = calculate_price_ratios(FUEL_PRICES_PATH, year=2024)
df_breakeven_for_d = compute_breakeven_cop(df_prices_for_d, afue_scenarios=[0.
    ↪90])
df_spark_for_d = compute_spark_gap_metrics(
    df_prices_for_d, cop_results[primary_mp],
    df_breakeven=df_breakeven_for_d, verbose=False,
)

header = (f"{'State':<6} {'COP':>6} {'Spark':>6} {'BE_90':>6} "
          f"{'Jenkins':>8} {'Strict':>7} {'Relaxed':>8} {'HP>BE?':>7}")
print(f"\n{header}")
print("-" * len(header))

n_strict = 0
n_relaxed = 0
for st, ref_val in sorted(jenkins_ref.items()):
    row_cop = cop_results[primary_mp][cop_results[primary_mp]['state'] == st]
    row_spark = df_spark_for_d[df_spark_for_d['state'] == st]

    if len(row_cop) == 0 or len(row_spark) == 0:
        print(f"st:<6} {'N/A':>6} {'N/A':>6} {'N/A':>6} {ref_val:>8.2f} "
              f"{'-':>7} {'-':>8} {'-':>7}")
        continue

    thermal_cop = row_cop['thermal_cop'].values[0]
    spark_gap = row_spark['spark_gap'].values[0]
    # ASSUMPTION: breakeven_cop_90 = spark_gap * 0.90
    breakeven_cop_90 = spark_gap * 0.90

    cop_exceeds = thermal_cop > breakeven_cop_90
    strict = abs(ref_val - breakeven_cop_90) <= 0.05
    relaxed = abs(ref_val - breakeven_cop_90) <= 0.50
    n_strict += int(strict)
    n_relaxed += int(relaxed)

```

```

    print(f"{st:<6} {thermal_cop:>6.2f} {spark_gap:>6.2f} {breakeven_cop_90:>6.
↳2f} "
        f"{ref_val:>8.2f} {' ' if strict else ' ':>7} "
        f"{' ' if relaxed else ' ':>8} {'Yes' if cop_exceeds else 'No':>7}")

n_total = len(jenkins_ref)
print(f"\nStrict match (±0.05): {n_strict}/{n_total}")
print(f"Relaxed match (±0.50): {n_relaxed}/{n_total}")
print(f"\nNote: Remaining discrepancies arise from (1) gas heat content "
      f"(Jenkins: 1020, ours: 1038 BTU/cf)")

# Interpretation
print("\n--- Interpretation ---")
warm = ['FL']
cold = ['MN', 'AK']
for label, states in [('Warm', warm), ('Cold', cold)]:
    for st in states:
        row_cop = cop_results[primary_mp][cop_results[primary_mp]['state'] ==
↳st]
        row_spark = df_spark_for_d[df_spark_for_d['state'] == st]
        if len(row_cop) == 0 or len(row_spark) == 0:
            continue
        cop_val = row_cop['thermal_cop'].values[0]
        be_val = row_spark['spark_gap'].values[0] * 0.90
        exceeds = cop_val > be_val
        msg = "HP saves money on fuel" if exceeds else "HP does NOT beat
↳furnace on fuel cost"
        print(f"  {st} ({label}): COP={cop_val:.2f}, BE_90={be_val:.2f} →
↳{msg}")

print(f"\n TASK D COMPLETE")

```

```

=====
TASK D: BREAK-EVEN COP CROSS-VALIDATION (Jenkins et al., MP3)
=====

```

State	COP	Spark	BE_90	Jenkins	Strict	Relaxed	HP>BE?
AK	N/A	N/A	N/A	5.69	-	-	-
CA	3.05	5.08	4.57	4.49			No
FL	2.89	1.70	1.53	1.50			Yes
MA	2.01	4.08	3.67	3.60			No
MN	1.59	4.41	3.97	3.90			No
PA	2.02	3.97	3.57	3.51			No

Strict match (±0.05): 1/6

Relaxed match (±0.50): 5/6

Note: Remaining discrepancies arise from (1) gas heat content (Jenkins: 1020, ours: 1038 BTU/cf)

--- Interpretation ---

FL (Warm): COP=2.89, BE_90=1.53 → HP saves money on fuel

MN (Cold): COP=1.59, BE_90=3.97 → HP does NOT beat furnace on fuel cost

TASK D COMPLETE

Step 4: Break-Even COP and Merged Metrics

```
[10]: print("=" * 80)
print("STEP 4: BREAK-EVEN COP & MERGED METRICS")
print("=" * 80)

# --- 4a: Compute break-even COP for multiple AFUE scenarios ---
# ASSUMPTION: 2024 EIA state-level residential prices.
afue_scenarios = [0.80, 0.90, 0.95, 1.00]
df_breakeven = compute_breakeven_cop(df_prices_csv,
    ↪afue_scenarios=afue_scenarios)
print(f" Break-even COP computed for {len(df_breakeven)} states, "
      f"AFUE scenarios: {afue_scenarios}")

print(f"\nBreak-even COP @90% AFUE - "
      f"Mean: {df_breakeven['breakeven_cop_90'].mean():.2f}, "
      f"Range: {df_breakeven['breakeven_cop_90'].min():.2f}-"
      f"{df_breakeven['breakeven_cop_90'].max():.2f}")

# --- 4b: Merge prices + COP + break-even into single state-level table ---
df_spark = compute_spark_gap_metrics(
    df_prices_csv, df_cop, df_breakeven=df_breakeven, verbose=True
)

# --- 4c: Validate against Jenkins Heat Pump Map reference values (90% AFUE) ---
# Source: Jenkins et al., break-even COP at 90% AFUE
# Note: Jenkins assumes 1020 BTU/cf heat content (ours: 1038) and a different
# price year. Differences are expected and documented in Task D.
jenkins_ref = {
    'FL': 1.50, 'PA': 3.51, 'MN': 3.90,
    'AK': 5.69, 'CA': 4.49, 'MA': 3.60,
}
print("\n--- Jenkins Validation (break-even COP @90% AFUE, ±0.05) ---")
print(f"{'State':<6} {'Computed':>10} {'Jenkins':>10} {'Diff':>8} {'Pass':>6}")
print("-" * 44)
for st, ref_val in sorted(jenkins_ref.items()):
    row = df_spark[df_spark['state'] == st]
```

```

if len(row) == 0:
    print(f"{st:<6} {'N/A':>10} {ref_val:>10.2f} {'-':>8} {'SKIP':>6}")
    continue
computed = row['breakeven_cop_90'].values[0]
diff = computed - ref_val
passed = abs(diff) <= 0.05
print(f"{st:<6} {computed:>10.2f} {ref_val:>10.2f} {diff:>+8.2f} {' ' if
↳passed else ' ':>6}")

# --- 4d: Cross-validation (effective COP vs break-even COP) ---
print("\n--- Cross-Validation: Effective COP vs Break-Even @90% ---")
warm_states = ['FL', 'GA', 'SC', 'TX', 'LA']
cold_states = ['MN', 'WI', 'VT', 'ND', 'ME']
for label, states in [('Warm', warm_states), ('Cold', cold_states)]:
    subset = df_spark[df_spark['state'].isin(states)].copy()
    if len(subset) == 0:
        continue
    subset['cop_minus_be'] = subset['thermal_cop'] - subset['breakeven_cop_90']
    print(f"\n{label} states:")
    print(f"  {'State':<6} {'COP':>6} {'BE_90':>6} {'Diff':>8} {'HP Wins?':
↳>10}")
    for _, r in subset.iterrows():
        hp_wins = r['thermal_cop'] > r['breakeven_cop_90']
        print(f"  {r['state']:<6} {r['thermal_cop']:>6.2f} "
              f"{r['breakeven_cop_90']:>6.2f} {r['cop_minus_be']:>+8.2f} "
              f"{'Yes' if hp_wins else 'No':>10}")

print(f"\n STEP 4 COMPLETE")

```

STEP 4: BREAK-EVEN COP & MERGED METRICS

Break-even COP computed for 51 states, AFUE scenarios: [0.8, 0.9, 0.95, 1.0]

Break-even COP @90% AFUE - Mean: 3.19, Range: 1.53-5.80

--- Spark Gap Metrics Summary (49 states) ---

Break-even COP @90% AFUE - Mean: 3.16, Range: 1.53-4.91

States where effective COP < break-even @90%: 35/49

--- Jenkins Validation (break-even COP @90% AFUE, ±0.05) ---

State	Computed	Jenkins	Diff	Pass
AK	N/A	5.69	-	SKIP
CA	4.57	4.49	+0.08	
FL	1.53	1.50	+0.03	
MA	3.67	3.60	+0.07	

MN	3.97	3.90	+0.07
PA	3.57	3.51	+0.06

--- Cross-Validation: Effective COP vs Break-Even @90% ---

Warm states:

State	COP	BE_90	Diff	HP Wins?
SC	2.60	2.30	+0.30	Yes
TX	2.75	2.28	+0.47	Yes
GA	2.60	2.12	+0.48	Yes
LA	2.76	1.96	+0.80	Yes
FL	2.89	1.53	+1.36	Yes

Cold states:

State	COP	BE_90	Diff	HP Wins?
WI	1.70	4.64	-2.94	No
MN	1.59	3.97	-2.38	No
ME	1.65	3.51	-1.86	No
ND	1.53	3.50	-1.97	No
VT	1.73	3.46	-1.73	No

STEP 4 COMPLETE

```
[16]: # Build COP & break-even summary table: state, spark gap, MP3 COP, MP4 COP, BE_
      ↪80/90/95
df_cop_be_summary = df_spark[['state', 'state_name', 'spark_gap',
                              'breakeven_cop_80', 'breakeven_cop_90',
                              ↪'breakeven_cop_95']].copy()

# Merge MP3 and MP4 thermal COP
for mp_num in [3, 4]:
    if mp_num in cop_results:
        df_cop_be_summary = df_cop_be_summary.merge(
            cop_results[mp_num][['state', 'thermal_cop']].rename(
                columns={'thermal_cop': f'thermal_cop_mp{mp_num}'}
            ),
            on='state', how='left',
        )

# Reorder columns for readability
col_order = ['state', 'state_name', 'spark_gap']
for mp_num in [3, 4]:
    col_name = f'thermal_cop_mp{mp_num}'
    if col_name in df_cop_be_summary.columns:
        col_order.append(col_name)
col_order += ['breakeven_cop_80', 'breakeven_cop_90', 'breakeven_cop_95']
```

```

df_cop_be_summary = df_cop_be_summary[col_order].sort_values('spark_gap',
    ↪ascending=False).reset_index(drop=True)

print("=" * 110)
print("THERMAL COP & BREAK-EVEN COP SUMMARY: TOP 5 & BOTTOM 5 STATES")
print("=" * 110)
print(f"\n{'Spark Gap & Break-Even COP based on 2024 EIA prices'}")
print(f"{'Break-Even COP = Spark Gap × AFUE (COP threshold where HP matches_"}
    ↪furnace fuel cost)'}\n")

print("--- Top 5 (highest spark gap) ---")
print(df_cop_be_summary.head(5).to_string(index=False))

print("\n--- Bottom 5 (lowest spark gap) ---")
print(df_cop_be_summary.tail(5).to_string(index=False))

# Highlight states where MP COP exceeds break-even at each AFUE
for mp_num in [3, 4]:
    cop_col = f'thermal_cop_mp{mp_num}'
    if cop_col not in df_cop_be_summary.columns:
        continue
    for afue_pct in [80, 90, 95]:
        be_col = f'breakeven_cop_{afue_pct}'
        n_beats = (df_cop_be_summary[cop_col] > df_cop_be_summary[be_col]).sum()
        n_total = df_cop_be_summary[cop_col].notna().sum()
        print(f"  MP{mp_num} COP > BE @{afue_pct}% AFUE: {n_beats}/{n_total}_
    ↪states")

```

```

=====
=====
THERMAL COP & BREAK-EVEN COP SUMMARY: TOP 5 & BOTTOM 5 STATES
=====
=====

```

Spark Gap & Break-Even COP based on 2024 EIA prices
 Break-Even COP = Spark Gap × AFUE (COP threshold where HP matches furnace fuel cost)

```

--- Top 5 (highest spark gap) ---
state state_name spark_gap thermal_cop_mp3 thermal_cop_mp4 breakeven_cop_80
breakeven_cop_90 breakeven_cop_95
  MI    Michigan      5.46      1.785815      2.829020      4.37
4.91      5.19
  CT Connecticut      5.20      2.085552      3.594760      4.16
4.68      4.94
  WI    Wisconsin      5.15      1.699153      2.623592      4.12
4.64      4.89

```


CA	California	5.08	3.054755	5.719542	4.06
4.57		4.83			
NM	New Mexico	4.77	2.384277	4.107143	3.82
4.29		4.53			

--- Bottom 5 (lowest spark gap) ---

state	state_name	spark_gap	thermal_cop_mp3	thermal_cop_mp4	breakeven_cop_80
breakeven_cop_90		breakeven_cop_95			
MO	Missouri	2.33	2.227515	4.083079	1.86
2.10		2.21			
AR	Arkansas	2.33	2.499449	4.830257	1.86
2.10		2.21			
AZ	Arizona	2.22	2.588321	4.684796	1.78
2.00		2.11			
LA	Louisiana	2.18	2.758595	5.345519	1.74
1.96		2.07			
FL	Florida	1.70	2.893052	5.697018	1.36
1.53		1.62			

MP3 COP > BE @80% AFUE: 18/49 states

MP3 COP > BE @90% AFUE: 14/49 states

MP3 COP > BE @95% AFUE: 11/49 states

MP4 COP > BE @80% AFUE: 34/49 states

MP4 COP > BE @90% AFUE: 29/49 states

MP4 COP > BE @95% AFUE: 27/49 states

Step 5: Geospatial Visualization

```
[11]: gdf_conus = None
      gdf_alaska = None

      try:
          print(f"Loading shapefile from: {SHAPEFILE_PATH}")
          gdf_states_raw = gpd.read_file(SHAPEFILE_PATH)
          _, gdf_conus, gdf_alaska = prepare_state_geodataframe(gdf_states_raw,
          ↪df_spark, merge_col='state')
          print(f" CONUS: {len(gdf_conus)}, Alaska: {len(gdf_alaska)}")
          print(" Geodataframe prepared")
      except FileNotFoundError:
          print(f" Shapefile not found at {SHAPEFILE_PATH} - skipping maps")
      except Exception as e:
          print(f" Could not load shapefile: {e}")
```

Loading shapefile from: c:\users\jorda\desktop\projects\cmu-tare-model\cmu_tare_model\data\electricity_ng_price_ratio\nhgis0011_shapefile_tl2015_us_state_2015\US_state_2015.shp

CONUS: 49, Alaska: 0

Geodataframe prepared

```
[12]: if gdf_conus is not None and gdf_alaska is not None:
    print("Generating choropleth maps...")

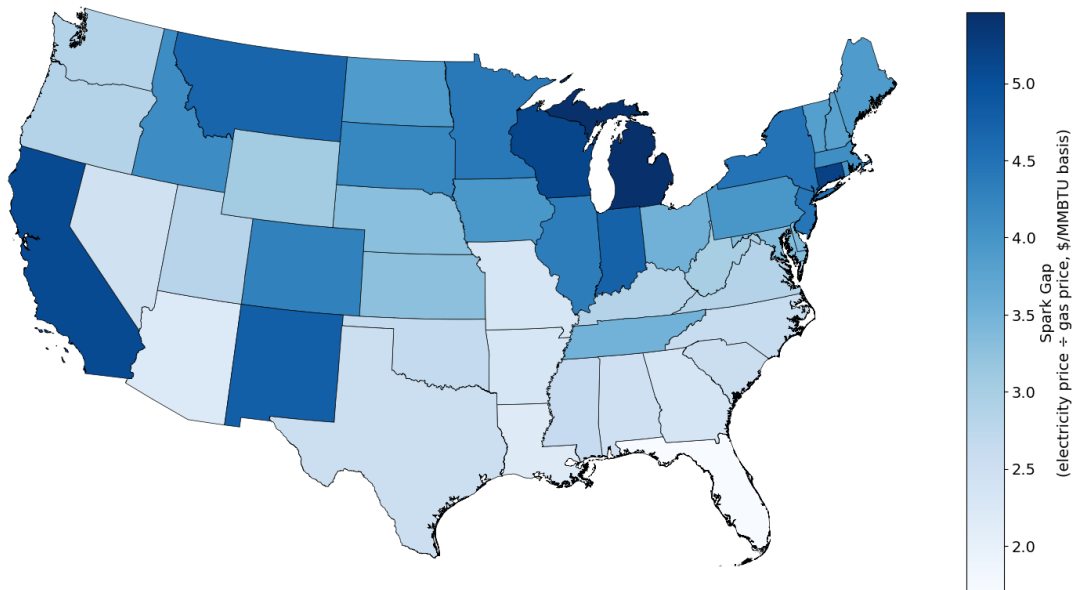
    # --- Spark gap (blue) ---
    create_choropleth_map(
        gdf_conus, gdf_alaska, column='spark_gap',
        title='Electricity-to-Natural Gas Spark Gap by State (2024)',
        cbar_label='Spark Gap\n(electricity price ÷ gas price, $/MMBTU basis)',
        output_path=os.path.join(PROJECT_ROOT, "state_spark_gap_map_2024.png"),
        cmap='Blues', show_plot=True,
    )

else:
    print(" Maps skipped - no geodataframe available")
```

Generating choropleth maps...

Saved: c:\users\jorda\desktop\projects\cmu-tare-model\state_spark_gap_map_2024.png

Electricity-to-Natural Gas Spark Gap by State (2024)



```
[13]: if gdf_conus is not None and gdf_alaska is not None:
    print("Generating choropleth maps...")

    # --- Thermal COP: MP3 top / MP4 bottom with shared color scale (greens) ---
    cop_mps = [mp for mp in [3, 4] if mp in cop_results]
    if len(cop_mps) >= 2:
```

```

# Build per-MP geodataframes and find shared color range
gdf_cop_panels = {}
all_vals = pd.Series(dtype=float)
for mp_num in cop_mps:
    df_spark_mp = compute_spark_gap_metrics(df_prices_csv,
↳ cop_results[mp_num])
    _, gdf_c, gdf_a = prepare_state_geodataframe(
        gdf_states_raw, df_spark_mp, merge_col='state'
    )
    gdf_cop_panels[mp_num] = (gdf_c, gdf_a)
    all_vals = pd.concat([all_vals, gdf_c['thermal_cop'],
↳ gdf_a['thermal_cop']])

    shared_norm = mcolors.Normalize(vmin=all_vals.dropna().min(),
↳ vmax=all_vals.dropna().max())
    col = 'thermal_cop'
    nrows = len(cop_mps)

    fig, axes_grid = plt.subplots(nrows, 1, figsize=(14, 9 * nrows),
↳ facecolor='white')
    if nrows == 1:
        axes_grid = [axes_grid]

    for i, mp_num in enumerate(cop_mps):
        gdf_c, gdf_a = gdf_cop_panels[mp_num]
        ax = axes_grid[i]

        gdf_c.plot(ax=ax, column=col, cmap='Greens', edgecolor='black',
                    linewidth=0.5, norm=shared_norm, legend=False)
        ax.set_axis_off()
        ax.set_title(f'MP{mp_num} - Thermal COP by State (2024)',
                    fontsize=18, fontweight='bold', pad=12)

        # Alaska inset - only plot if data exists
        inset_bounds = ax.get_position()
        ax_ak = fig.add_axes([
            inset_bounds.x0, inset_bounds.y0,
            inset_bounds.width * 0.22, inset_bounds.height * 0.28,
        ])
        if not gdf_a.empty:
            gdf_a.plot(ax=ax_ak, column=col, cmap='Greens',
↳ edgecolor='black',
                    linewidth=0.5, norm=shared_norm, legend=False)
            ax_ak.set_axis_off()

        # Shared colorbar - bottom center
        cax = fig.add_axes([0.25, 0.02, 0.50, 0.015])

```

```

sm = plt.cm.ScalarMappable(cmap='Greens', norm=shared_norm)
sm.set_array([])
cbar = fig.colorbar(sm, cax=cax, orientation='horizontal')
cbar.set_label('Thermal COP (heat delivered / electricity consumed)',
↳ fontsize=16)
cax.tick_params(labelsize=16)

fig.subplots_adjust(hspace=0.04)
output = os.path.join(PROJECT_ROOT, "state_thermal_cop_mp3_mp4_map_2024.
↳ png")
fig.savefig(output, dpi=600, bbox_inches='tight', facecolor='white')
print(f" Saved: {output}")
plt.show()
elif len(cop_mps) == 1:
    create_choropleth_map(
        gdf_conus, gdf_alaska, column='thermal_cop',
        title=f'Heat Pump Thermal COP by State (MP{cop_mps[0]}, 2024)',
        cbar_label='Thermal COP\n(heat delivered / electricity consumed)',
        output_path=os.path.join(PROJECT_ROOT,
↳ f"state_thermal_cop_mp{cop_mps[0]}_map_2024.png"),
        cmap='Greens', show_plot=True,
    )
else:
    print(" No thermal COP results for MP3 or MP4 - skipping COP maps")

    print(" Maps generated")
else:
    print(" Maps skipped - no geodataframe available")

```

Generating choropleth maps...

Saved: c:\users\jorda\desktop\projects\cmu-tare-
model\state_thermal_cop_mp3_mp4_map_2024.png



Maps generated

[]:

```
[14]: from typing import Optional, Tuple
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

def create_choropleth_map(
    gdf_conus: gpd.GeoDataFrame,
    gdf_alaska: gpd.GeoDataFrame,
    column: str,
    title: str,
    cbar_label: str,
    year: int = 2024,
    output_path: Optional[str] = None,
    figsize: Tuple[int, int] = (16, 10),
    dpi: int = 600,
    cmap: str = 'Blues',
    norm=None,
    show_plot: bool = True
) -> Optional[str]:
    """
    Create a choropleth map with CONUS main panel and Alaska inset.

    Pass a matplotlib Normalize (e.g. TwoSlopeNorm) via `norm` for diverging
    maps; otherwise linear normalization is derived from the data automatically.
    """
    plot_kw = dict(column=column, cmap=cmap, edgecolor='black', linewidth=0.5,
    ↪ legend=False)

    if norm is not None:
        plot_kw['norm'] = norm
    else:
        vals = pd.concat([gdf_conus[column], gdf_alaska[column]]).dropna()
        norm = mcolors.Normalize(vmin=vals.min(), vmax=vals.max())
        plot_kw['vmin'], plot_kw['vmax'] = vals.min(), vals.max()

    fig = plt.figure(figsize=figsize, facecolor='white')
    ax_main = fig.add_axes([0.02, 0.02, 0.78, 0.90])
    ax_ak = fig.add_axes([0.02, 0.02, 0.22, 0.25])
```

```

gdf_conus.plot(ax=ax_main, **plot_kw)
ax_main.set_axis_off()
ax_main.set_title(title, fontsize=20, fontweight='bold', pad=12)

if not gdf_alaska.empty:
    gdf_alaska.plot(ax=ax_ak, **plot_kw)
ax_ak.set_axis_off()

# Colorbar - matplotlib handles tick placement natively for any norm
cax = fig.add_axes([0.82, 0.08, 0.03, 0.74])
sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
sm.set_array([])
fig.colorbar(sm, cax=cax, label=cbar_label).ax.yaxis.label.set_fontsize(14)
cax.tick_params(labelsize=14)

if output_path:
    fig.savefig(output_path, dpi=dpi, bbox_inches='tight',
        facecolor='white')
    print(f" Saved: {output_path}")
if show_plot:
    plt.show()
else:
    plt.close(fig)
return output_path if output_path else None

if gdf_conus is not None and gdf_alaska is not None:
    print("Generating choropleth maps...")

    # --- Break-even COP at 90% AFUE (green) ---
    create_choropleth_map(
        gdf_conus, gdf_alaska, column='breakeven_cop_90',
        title='Break-Even COP by State at 90% AFUE (2024)',
        cbar_label='Break-Even COP\n(COP needed for HP to match furnace cost)',
        output_path=os.path.join(PROJECT_ROOT, "state_breakeven_cop_90_map_2024.
        png"),
        cmap='Greens', show_plot=True,
    )

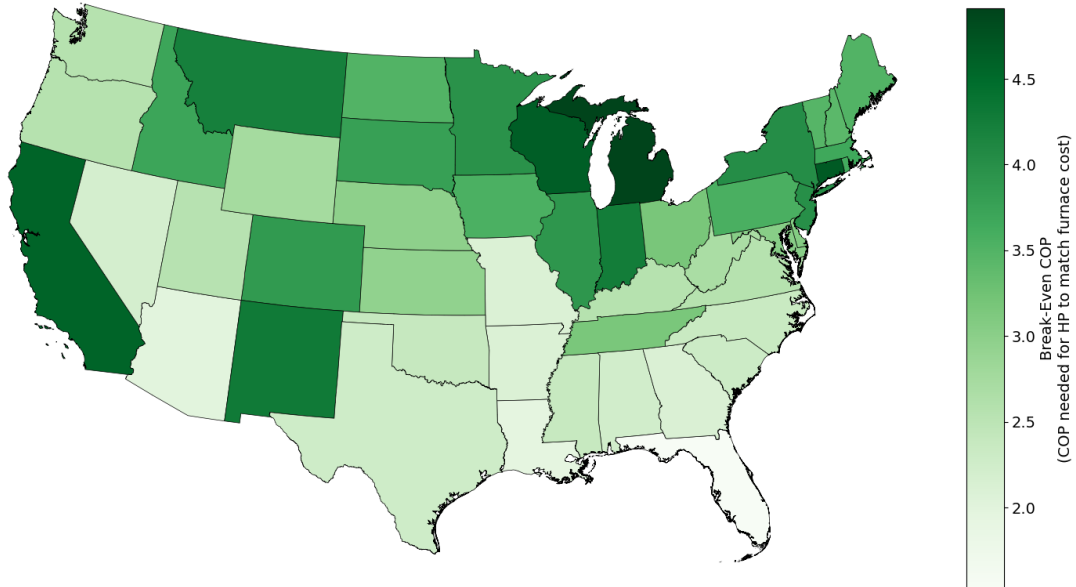
    print(" Maps generated")
else:
    print(" Maps skipped - no geodataframe available")

```

Generating choropleth maps...

Saved: c:\users\jorda\desktop\projects\cmu-tare-model\state_breakeven_cop_90_map_2024.png

Break-Even COP by State at 90% AFUE (2024)



Maps generated

```
[17]: if gdf_conus is not None and gdf_alaska is not None:
    print("Generating break-even COP panel maps (80%, 90%, 95% AFUE)...")

    be_cols = ['breakeven_cop_80', 'breakeven_cop_90', 'breakeven_cop_95']
    be_labels = ['80% AFUE', '90% AFUE', '95% AFUE']

    # Shared color range across all three AFUE scenarios
    all_be_vals = pd.concat([
        pd.concat([gdf_conus[c], gdf_alaska[c]] for c in be_cols
    ]).dropna()
    shared_norm = mcolors.Normalize(vmin=all_be_vals.min(), vmax=all_be_vals.
    ↪max())

    nrows = len(be_cols)
    fig, axes_grid = plt.subplots(nrows, 1, figsize=(14, 9 * nrows),
    ↪facecolor='white')

    for i, (col, label) in enumerate(zip(be_cols, be_labels)):
        ax = axes_grid[i]
        gdf_conus.plot(ax=ax, column=col, cmap='Oranges', edgecolor='black',
                        linewidth=0.5, norm=shared_norm, legend=False)
        ax.set_axis_off()
```



```

ax.set_title(f'Break-Even COP by State - {label} (2024)',
             fontsize=18, fontweight='bold', pad=12)

# Alaska inset
inset_bounds = ax.get_position()
ax_ak = fig.add_axes([
    inset_bounds.x0, inset_bounds.y0,
    inset_bounds.width * 0.22, inset_bounds.height * 0.28,
])
if not gdf_alaska.empty:
    gdf_alaska.plot(ax=ax_ak, column=col, cmap='Oranges',
                    edgecolor='black',
                    linewidth=0.5, norm=shared_norm, legend=False)
    ax_ak.set_axis_off()

# Shared colorbar - bottom center
cax = fig.add_axes([0.25, 0.02, 0.50, 0.015])
sm = plt.cm.ScalarMappable(cmap='Oranges', norm=shared_norm)
sm.set_array([])
cbar = fig.colorbar(sm, cax=cax, orientation='horizontal')
cbar.set_label('Break-Even COP (COP threshold for HP to match furnace fuel_
cost)', fontsize=16)
cax.tick_params(labelsize=16)

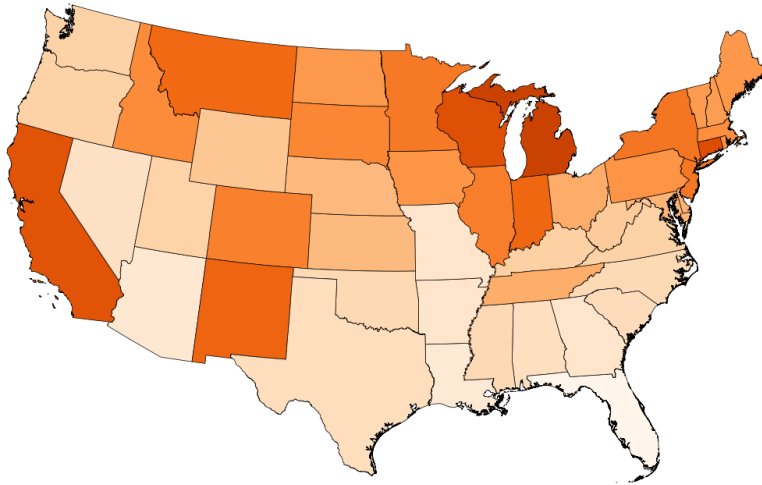
fig.subplots_adjust(hspace=0.04)
output = os.path.join(PROJECT_ROOT,
"state_breakeven_cop_80_90_95_panel_map_2024.png")
fig.savefig(output, dpi=600, bbox_inches='tight', facecolor='white')
print(f" Saved: {output}")
plt.show()
else:
    print(" Maps skipped - no geodataframe available")

```

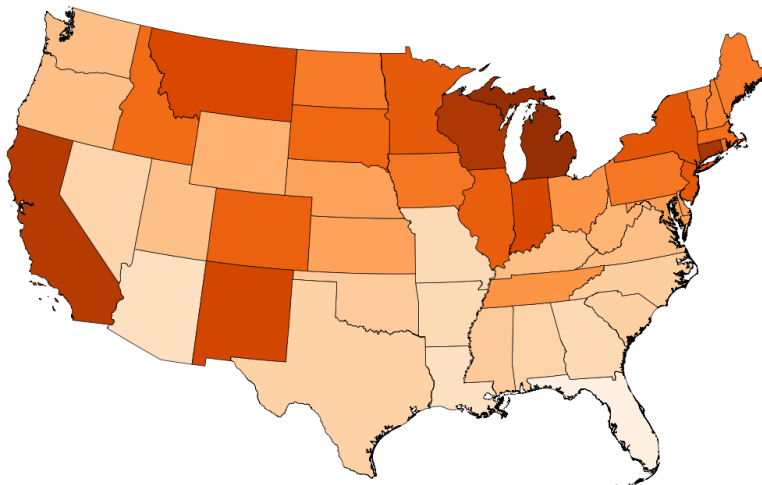
Generating break-even COP panel maps (80%, 90%, 95% AFUE)...

Saved: c:\users\jorda\desktop\projects\cmu-tare-
model\state_breakeven_cop_80_90_95_panel_map_2024.png

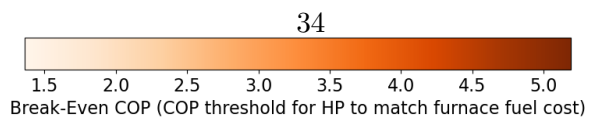
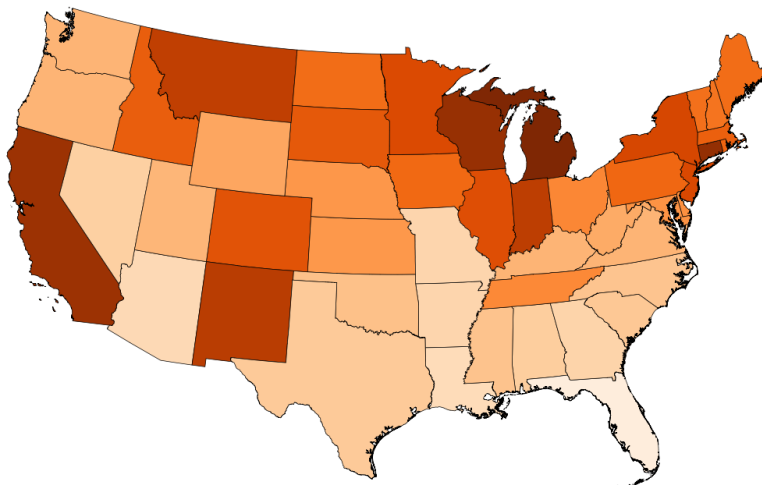
Break-Even COP by State — 80% AFUE (2024)



Break-Even COP by State — 90% AFUE (2024)



Break-Even COP by State — 95% AFUE (2024)



Display Results

```
[15]: print("==== STATE PRICE RATIOS (2024 nominal) =====\n")
display(df_prices_csv)

for mp in selected_mps:
    print(f"\n==== THERMAL COP & AFUE (MP{mp}, NG homes) =====\n")
    display(cop_results[mp].sort_values('thermal_cop', ascending=False)[
        ['state', 'thermal_cop', 'baseline_afue', 'fans_pumps_pct',
        ↪ 'home_count']
    ])

# --- Comparison table: spark gap, break-even COP, effective COP ---
print(f"\n==== SPARK GAP & BREAK-EVEN COP vs EFFECTIVE COP (MP{primary_mp})\n")
    ↪=====)

df_display = df_spark[[
    'state', 'state_name', 'spark_gap',
    'breakeven_cop_80', 'breakeven_cop_90', 'breakeven_cop_95',
    ↪ 'breakeven_cop_100',
    'thermal_cop', 'baseline_afue',
]] .copy()

# Flag whether the heat pump beats the break-even threshold at 90% AFUE
df_display['hp_beats_breakeven_90'] = df_display['thermal_cop'] >
    ↪ df_display['breakeven_cop_90']

display(df_display.sort_values('spark_gap', ascending=False).
    ↪ reset_index(drop=True))
```

==== STATE PRICE RATIOS (2024 nominal) =====

	state	state_name	elec_price_kwh	gas_price_kwh	\
0	AK	Alaska	0.2482	0.0386	
1	MI	Michigan	0.1930	0.0354	
2	CT	Connecticut	0.2875	0.0553	
3	WI	Wisconsin	0.1718	0.0333	
4	CA	California	0.3197	0.0629	
5	NM	New Mexico	0.1420	0.0297	
6	IN	Indiana	0.1477	0.0312	
7	MT	Montana	0.1266	0.0270	
8	NY	New York	0.2443	0.0545	
9	NJ	New Jersey	0.1934	0.0434	
10	MN	Minnesota	0.1545	0.0350	

11	IL	Illinois	0.1587	0.0365
12	CO	Colorado	0.1492	0.0348
13	SD	South Dakota	0.1286	0.0306
14	ID	Idaho	0.1152	0.0280
15	MA	Massachusetts	0.2935	0.0720
16	RI	Rhode Island	0.2865	0.0712
17	PA	Pennsylvania	0.1777	0.0448
18	IA	Iowa	0.1340	0.0340
19	ME	Maine	0.2429	0.0623
20	ND	North Dakota	0.1151	0.0296
21	VT	Vermont	0.2190	0.0570
22	NH	New Hampshire	0.2340	0.0616
23	OH	Ohio	0.1599	0.0453
24	TN	Tennessee	0.1242	0.0353
25	DE	Delaware	0.1657	0.0486
26	MD	Maryland	0.1786	0.0531
27	NE	Nebraska	0.1153	0.0349
28	KS	Kansas	0.1415	0.0432
29	DC	District of Columbia	0.1771	0.0542
30	WY	Wyoming	0.1247	0.0407
31	WV	West Virginia	0.1507	0.0502
32	KY	Kentucky	0.1279	0.0447
33	WA	Washington	0.1190	0.0418
34	VA	Virginia	0.1441	0.0506
35	OR	Oregon	0.1470	0.0516
36	UT	Utah	0.1222	0.0432
37	HI	Hawaii	0.4286	0.1607
38	OK	Oklahoma	0.1224	0.0460
39	MS	Mississippi	0.1339	0.0507
40	NC	North Carolina	0.1413	0.0540
41	SC	South Carolina	0.1423	0.0556
42	TX	Texas	0.1494	0.0591
43	AL	Alabama	0.1518	0.0612
44	NV	Nevada	0.1500	0.0614
45	GA	Georgia	0.1408	0.0598
46	MO	Missouri	0.1291	0.0554
47	AR	Arkansas	0.1232	0.0529
48	AZ	Arizona	0.1491	0.0673
49	LA	Louisiana	0.1173	0.0538
50	FL	Florida	0.1414	0.0834

	elec_price_mmbtu	gas_price_mmbtu	spark_gap
0	72.74	11.30	6.44
1	56.56	10.37	5.46
2	84.26	16.21	5.20
3	50.35	9.77	5.15
4	93.69	18.44	5.08
5	41.62	8.72	4.77

6	43.29	9.15	4.73
7	37.10	7.92	4.69
8	71.60	15.97	4.48
9	56.68	12.73	4.45
10	45.28	10.26	4.41
11	46.51	10.70	4.35
12	43.73	10.19	4.29
13	37.69	8.98	4.20
14	33.76	8.20	4.12
15	86.02	21.10	4.08
16	83.96	20.87	4.02
17	52.08	13.12	3.97
18	39.27	9.95	3.95
19	71.19	18.26	3.90
20	33.73	8.66	3.89
21	64.18	16.70	3.84
22	68.58	18.05	3.80
23	46.86	13.27	3.53
24	36.40	10.36	3.51
25	48.56	14.24	3.41
26	52.34	15.55	3.37
27	33.79	10.24	3.30
28	41.47	12.66	3.28
29	51.90	15.89	3.27
30	36.55	11.94	3.06
31	44.17	14.70	3.00
32	37.48	13.11	2.86
33	34.88	12.25	2.85
34	42.23	14.84	2.85
35	43.08	15.13	2.85
36	35.81	12.67	2.83
37	125.61	47.09	2.67
38	35.87	13.48	2.66
39	39.24	14.85	2.64
40	41.41	15.82	2.62
41	41.70	16.29	2.56
42	43.78	17.32	2.53
43	44.49	17.95	2.48
44	43.96	18.01	2.44
45	41.26	17.52	2.35
46	37.84	16.24	2.33
47	36.11	15.50	2.33
48	43.70	19.72	2.22
49	34.38	15.78	2.18
50	41.44	24.44	1.70

===== THERMAL COP & AFUE (MP3, NG homes) =====

	state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
3	CA	3.054755	0.801801	4.001501	24571
8	FL	2.893052	0.794453	5.682569	1084
16	LA	2.758595	0.799742	5.761571	2139
41	TX	2.748806	0.804379	5.035227	12362
6	DC	2.715392	0.752035	4.055966	338
35	OR	2.618561	0.820170	3.725680	2183
0	AL	2.608147	0.805110	4.792433	1853
38	SC	2.603092	0.808995	4.670975	1580
9	GA	2.598895	0.812388	4.839350	5111
2	AZ	2.588321	0.823168	5.414082	2757
23	MS	2.584149	0.810445	5.177692	1219
45	WA	2.510266	0.817851	3.375780	3540
1	AR	2.499449	0.804689	5.161718	1582
34	OK	2.493255	0.806185	4.651168	2872
31	NV	2.491338	0.821705	4.966447	1998
25	NC	2.471322	0.807238	4.645363	3479
40	TN	2.407582	0.802505	4.752710	3023
18	MD	2.398846	0.789437	4.136133	3115
30	NM	2.384277	0.811805	5.304953	1506
43	VA	2.380144	0.801493	4.688277	3468
7	DE	2.314899	0.802174	4.076548	533
29	NJ	2.294246	0.795664	3.462339	6742
15	KY	2.290071	0.785014	4.005665	2261
22	MO	2.227515	0.795452	4.501767	4302
14	KS	2.200695	0.787260	3.997958	2599
42	UT	2.162580	0.821928	4.524524	2505
11	ID	2.154783	0.824162	4.403011	1118
5	CT	2.085552	0.803036	2.979604	1141
47	WV	2.051419	0.797393	2.992868	1076
36	PA	2.020353	0.795560	2.898380	8594
17	MA	2.012107	0.800042	2.691154	3090
33	OH	1.989596	0.803509	2.943619	10373
48	WY	1.977422	0.807016	4.351049	423
32	NY	1.974215	0.793466	2.439740	8652
13	IN	1.971601	0.804715	3.235824	5434
4	CO	1.952172	0.823187	4.292325	4566
12	IL	1.945797	0.804501	3.148780	11086
37	RI	1.936985	0.802848	2.434246	477
27	NE	1.897562	0.807015	3.166600	1507
24	MT	1.892323	0.809666	3.418760	705
28	NH	1.853915	0.819283	2.335242	232
20	MI	1.785815	0.809416	2.402143	9884
10	IA	1.763096	0.806999	3.002361	2637
44	VT	1.731443	0.805144	1.858951	110
46	WI	1.699153	0.804310	2.207530	4759

39	SD	1.673837	0.797856	2.435159	520
19	ME	1.647215	0.819947	1.661832	61
21	MN	1.589963	0.801746	1.937747	4687
26	ND	1.527465	0.803502	1.730071	402

===== THERMAL COP & AFUE (MP4, NG homes) =====

	state	thermal_cop	baseline_afue	fans_pumps_pct	home_count
3	CA	5.719542	0.801801	5.075078	24571
8	FL	5.697018	0.794453	5.459375	1084
16	LA	5.345519	0.799742	5.496742	2139
41	TX	5.303021	0.804379	5.558138	12362
0	AL	5.058210	0.805110	5.564781	1853
9	GA	5.057374	0.812388	5.699106	5111
38	SC	5.056874	0.808995	5.632238	1580
23	MS	5.024481	0.810445	5.541313	1219
6	DC	4.980642	0.752035	4.335234	338
1	AR	4.830257	0.804689	5.518244	1582
34	OK	4.799527	0.806185	5.577920	2872
25	NC	4.723897	0.807238	5.484151	3479
2	AZ	4.684796	0.823168	5.024505	2757
35	OR	4.661229	0.820170	5.222965	2183
40	TN	4.567381	0.802505	5.355325	3023
31	NV	4.481222	0.821705	5.013914	1998
43	VA	4.460033	0.801493	5.207943	3468
18	MD	4.428351	0.789437	4.937560	3115
7	DE	4.261080	0.802174	4.993519	533
45	WA	4.219903	0.817851	4.817898	3540
15	KY	4.146416	0.785014	4.782815	2261
29	NJ	4.130790	0.795664	4.734180	6742
30	NM	4.107143	0.811805	4.772070	1506
22	MO	4.083079	0.795452	5.001774	4302
14	KS	3.980038	0.787260	4.902813	2599
42	UT	3.697648	0.821928	4.869321	2505
5	CT	3.594760	0.803036	4.334917	1141
11	ID	3.594294	0.824162	4.885474	1118
47	WV	3.435717	0.797393	4.085933	1076
17	MA	3.374642	0.800042	4.013928	3090
33	OH	3.343245	0.803509	4.154727	10373
36	PA	3.341868	0.795560	3.909586	8594
13	IN	3.337385	0.804715	4.238571	5434
12	IL	3.301733	0.804501	4.245725	11086
32	NY	3.201888	0.793466	3.668657	8652
27	NE	3.179736	0.807015	4.236552	1507
37	RI	3.166801	0.802848	3.811201	477
4	CO	3.075468	0.823187	3.944798	4566
28	NH	3.055892	0.819283	3.936944	232

48	WY	2.960037	0.807016	3.671006	423
10	IA	2.849892	0.806999	3.731092	2637
24	MT	2.845549	0.809666	3.562685	705
20	MI	2.829020	0.809416	3.567197	9884
44	VT	2.676052	0.805144	3.111348	110
46	WI	2.623592	0.804310	3.174381	4759
39	SD	2.502694	0.797856	3.040386	520
19	ME	2.444452	0.819947	2.846700	61
21	MN	2.359931	0.801746	2.772684	4687
26	ND	2.149135	0.803502	2.371268	402

===== SPARK GAP & BREAK-EVEN COP vs EFFECTIVE COP (MP3) =====

	state	state_name	spark_gap	breakeven_cop_80	breakeven_cop_90	\
0	MI	Michigan	5.46	4.37	4.91	
1	CT	Connecticut	5.20	4.16	4.68	
2	WI	Wisconsin	5.15	4.12	4.64	
3	CA	California	5.08	4.06	4.57	
4	NM	New Mexico	4.77	3.82	4.29	
5	IN	Indiana	4.73	3.78	4.26	
6	MT	Montana	4.69	3.75	4.22	
7	NY	New York	4.48	3.58	4.03	
8	NJ	New Jersey	4.45	3.56	4.00	
9	MN	Minnesota	4.41	3.53	3.97	
10	IL	Illinois	4.35	3.48	3.91	
11	CO	Colorado	4.29	3.43	3.86	
12	SD	South Dakota	4.20	3.36	3.78	
13	ID	Idaho	4.12	3.30	3.71	
14	MA	Massachusetts	4.08	3.26	3.67	
15	RI	Rhode Island	4.02	3.22	3.62	
16	PA	Pennsylvania	3.97	3.18	3.57	
17	IA	Iowa	3.95	3.16	3.56	
18	ME	Maine	3.90	3.12	3.51	
19	ND	North Dakota	3.89	3.11	3.50	
20	VT	Vermont	3.84	3.07	3.46	
21	NH	New Hampshire	3.80	3.04	3.42	
22	OH	Ohio	3.53	2.82	3.18	
23	TN	Tennessee	3.51	2.81	3.16	
24	DE	Delaware	3.41	2.73	3.07	
25	MD	Maryland	3.37	2.70	3.03	
26	NE	Nebraska	3.30	2.64	2.97	
27	KS	Kansas	3.28	2.62	2.95	
28	DC	District of Columbia	3.27	2.62	2.94	
29	WY	Wyoming	3.06	2.45	2.75	
30	WV	West Virginia	3.00	2.40	2.70	
31	KY	Kentucky	2.86	2.29	2.57	
32	WA	Washington	2.85	2.28	2.56	

33	VA	Virginia	2.85	2.28	2.56
34	OR	Oregon	2.85	2.28	2.56
35	UT	Utah	2.83	2.26	2.55
36	OK	Oklahoma	2.66	2.13	2.39
37	MS	Mississippi	2.64	2.11	2.38
38	NC	North Carolina	2.62	2.10	2.36
39	SC	South Carolina	2.56	2.05	2.30
40	TX	Texas	2.53	2.02	2.28
41	AL	Alabama	2.48	1.98	2.23
42	NV	Nevada	2.44	1.95	2.20
43	GA	Georgia	2.35	1.88	2.12
44	MO	Missouri	2.33	1.86	2.10
45	AR	Arkansas	2.33	1.86	2.10
46	AZ	Arizona	2.22	1.78	2.00
47	LA	Louisiana	2.18	1.74	1.96
48	FL	Florida	1.70	1.36	1.53

	breakeven_cop_95	breakeven_cop_100	thermal_cop	baseline_afue	\
0	5.19	5.46	1.785815	0.809416	
1	4.94	5.20	2.085552	0.803036	
2	4.89	5.15	1.699153	0.804310	
3	4.83	5.08	3.054755	0.801801	
4	4.53	4.77	2.384277	0.811805	
5	4.49	4.73	1.971601	0.804715	
6	4.46	4.69	1.892323	0.809666	
7	4.26	4.48	1.974215	0.793466	
8	4.23	4.45	2.294246	0.795664	
9	4.19	4.41	1.589963	0.801746	
10	4.13	4.35	1.945797	0.804501	
11	4.08	4.29	1.952172	0.823187	
12	3.99	4.20	1.673837	0.797856	
13	3.91	4.12	2.154783	0.824162	
14	3.88	4.08	2.012107	0.800042	
15	3.82	4.02	1.936985	0.802848	
16	3.77	3.97	2.020353	0.795560	
17	3.75	3.95	1.763096	0.806999	
18	3.70	3.90	1.647215	0.819947	
19	3.70	3.89	1.527465	0.803502	
20	3.65	3.84	1.731443	0.805144	
21	3.61	3.80	1.853915	0.819283	
22	3.35	3.53	1.989596	0.803509	
23	3.33	3.51	2.407582	0.802505	
24	3.24	3.41	2.314899	0.802174	
25	3.20	3.37	2.398846	0.789437	
26	3.14	3.30	1.897562	0.807015	
27	3.12	3.28	2.200695	0.787260	
28	3.11	3.27	2.715392	0.752035	
29	2.91	3.06	1.977422	0.807016	

30	2.85	3.00	2.051419	0.797393
31	2.72	2.86	2.290071	0.785014
32	2.71	2.85	2.510266	0.817851
33	2.71	2.85	2.380144	0.801493
34	2.71	2.85	2.618561	0.820170
35	2.69	2.83	2.162580	0.821928
36	2.53	2.66	2.493255	0.806185
37	2.51	2.64	2.584149	0.810445
38	2.49	2.62	2.471322	0.807238
39	2.43	2.56	2.603092	0.808995
40	2.40	2.53	2.748806	0.804379
41	2.36	2.48	2.608147	0.805110
42	2.32	2.44	2.491338	0.821705
43	2.23	2.35	2.598895	0.812388
44	2.21	2.33	2.227515	0.795452
45	2.21	2.33	2.499449	0.804689
46	2.11	2.22	2.588321	0.823168
47	2.07	2.18	2.758595	0.799742
48	1.62	1.70	2.893052	0.794453

	hp_beats_breakeven_90
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	False
10	False
11	False
12	False
13	False
14	False
15	False
16	False
17	False
18	False
19	False
20	False
21	False
22	False
23	False
24	False
25	False
26	False

27	False
28	False
29	False
30	False
31	False
32	False
33	False
34	True
35	False
36	True
37	True
38	True
39	True
40	True
41	True
42	True
43	True
44	True
45	True
46	True
47	True
48	True